

Transcript

Module 10: Introduction to Backend Development

Video 1: Module Overview

Machine learning and deep learning models lie at the heart of modern Artificial Intelligence or AI systems. But equally important are the software applications that integrate these models seamlessly to automate business processes and decision making to create values. Software application development can be broadly divided into frontend development and backend development. Backend development mainly involves server-side rendering web applications and Application Programming Interface or API endpoints.

This module focuses on the former. In particular, this module will impart essential knowledge and skills on developing server-side rendering web applications to you. This is a comprehensive module that will explain how to develop web applications using two of the most popular programming languages, JavaScript and Python. For each programming language, a suitable framework will be taught to you. You will also learn how to create web pages with dynamic content retrieved from a database. Get ready to embark on yet another exciting learning journey with me and jump-start your career in backend development.

Video 2: Backend Software Engineering with JavaScript and Python

In this video, I will demystify the differences between backend and frontend software engineering with you. I will also explain the difference between client-side rendering and server-side rendering web application. Modern software applications manifest in many form factors such as desktop, web, and mobile applications. Web applications are among the most common ones.

Web applications run within a web browser and are developed with standard-based technologies such as Hypertext Markup Language or HTML, and Cascading Style Sheets or CSS. Examples of web applications include digital commerce websites, communication tools and even productivity software. Web applications are increasingly popular as compared to desktop applications for at least three reasons. First, they are cross-platform compatible and easily run across operating systems and devices. Second, they impose minimal requirements on the computer of end-users. Finally, they simplify deployment, installation, and maintenance.

Web applications are typically structured using the client-server architecture. The server contains the resources to be shared with multiple users or clients, in this case the web pages. The client, in this case a web browser, will request the required web pages from the server and display them to the user. This infographic shows the client-server architecture of general software applications. Development tasks associated with the data tier and logic tier of an application are typically known as backend development. Those that are associated with the presentation tier are known as frontend development. The data tier and logic tier typically run on the server whereas the presentation tier runs on the client. However, for web applications, the code for the presentation tier, which are the code for generating the HTML content, can be executed on either the server or client. The former is known as server-side HTML rendering and the latter is known as client-side HTML rendering.

In server-side rendering, each time the client requires a web page, it must send a request to the server to download the content. Thus, a user browsing a website will normally lead to multiple requests being sent to the server, one request for each web page.

In client-side rendering, the browser will download all the content for the entire website when the user first views the website. The client then renders each required web page within the web browser as the user navigates around the website. Typically, there are only a few requests made to the server.

Server-side rendering and client-side rendering each have its own pros and cons. Thus, you should choose the most appropriate approach for the website or web application that you are creating. For examples, digital commerce or content applications that rely on Search Engine Optimisation or SEO are generally more suited to server-side rendering. Applications such as data dashboard that require faster responsiveness and higher interactivity are more suited to client-side rendering. In server-side rendering, since the presentation tier code runs on the server, it exhibits a stronger notion of backend development. As for client-side rendering, it will certainly fall under frontend development.

JavaScript can be used for both backend development and frontend development. For backend development and frontend development involving server-side rendering web applications, Node.js and its associated frameworks such as Express.js can be used. As for client-side rendering web applications, popular JavaScript frameworks include React, Angular, and Vue. Python can also be used for both backend development and frontend development. But for frontend development, Python is only suitable for creating server-side rendering web applications. Popular Python web development frameworks include Django and Flask.

This module focuses on creating server-side rendering web applications using JavaScript with Express.js and Python with Flask. The subsequent videos in this module we will explain more about each of these two development stacks.

Video 3: Introduction to Node.js and Express.js

This video introduces you to the Node.js and Express.js frameworks for web application development using JavaScript. In the past, JavaScript only runs within a web browser and JavaScript is thus considered as client-side scripting. In this role, JavaScript is mainly used for implementing use cases within the web browser, such as form data validation, general web page interactivity and JavaScript games. However, with the arrival of Node.js, JavaScript can now run outside the web browser environment and can be used for general purpose programming.

This example depicts two sample codes written in JavaScript. The sample code on the left shows a JavaScript embedded within a typical HTML document for running within a web browser. The sample code on the right shows a standalone JavaScript outside of a HTML document for running in Node.js without a web browser. Although the two JavaScript codes look similar and perform essentially the same task of outputting a string message, the ability to run the sample code on the right outside of a web browser represents a quantum leap in the capability of JavaScript.

More specifically, Node.js is an open-source, cross-platform runtime environment for JavaScript. It can be used to create all kinds of software tools and applications in JavaScript. In particular, Node.js can be used for server-side backend development to create the data tier and logic tier of a web application. This is also known as server-side scripting. For this reason, the most common application of Node.js is a web server that listens for any kind of HTTP request. HTTP stands for the Hypertext Transfer Protocol and is the primary protocol for exchanging HTML content or webpages between a web browser and web server.

In this example, the Node.js application will respond to a HTTP GET request that is sent to the root path by returning a string containing "HELLO WORLD".

If you browse to this address in a web browser, you will see the web browser displaying the text "HELLO WORLD". The next example shows the Node.js application returning a string representing a well-formed HTML content or webpage which displays a greeting webpage. If you browse to this address in a web browser, you will now see a proper webpage displaying the text.

Although Node.js is a very powerful framework, it does not support many common web application development tasks directly. Let's examine some of these important tasks. First, Node.js does not allow the developer to easily add specific handling for different HTTP verbs such as GET, PUT, POST and DELETE at the same path or route. In the previous examples, we are only handling the GET verb at the root path by default. Second, it is difficult to handle HTTP request at different paths or routes within the same web application. More specifically, it is common to use different routes to identify different sections of a web application and this cannot be done easily with Node.js. For example, you might want to respond to the HTTP GET requests at the the /car and /book routes by displaying a list of cars and boots respectively that are available for sales on the website.

Instead of rendering HTML content as string data within the code, it is more intuitive and easier to markup the content in a proper template file. By separating the HTML tags from the code, it is easier to create and change web content, structure and styling. Using templates also make it easier to reuse CSS styling. But Node.js does not provide support for web page template directly. Fourth, most web applications need to handle some form of conversational state. Conversational state arises when the same user makes multiple HTTP requests to the server and the server needs to understand that these multiple requests are indeed from the same user. For example, in an e-commerce web application, when a user adds multiple product items to the shopping cart, the web application must ensure that all the items are correctly added to the user's shopping cart and not the shopping cart of another different user.

Of course, when the same user checkouts and makes payment, the customer should be paying for his or her purchase. These are just some limitations of Node.js. For these reasons and more, it is often preferable to use a full-fledged web application development framework that is built on top of Node.js.

Express.js is the most popular Node.js web framework available. In this example, the Express.js application can respond to the HTTP GET requests that are sent to the /car and /book routes easily. If you browse to this address and this address in the web browser, you will be able to see a proper web page displaying a list of cars and books, respectively. Express.js provides mechanisms to support many common web development tasks that enhance the productivity and efficacy of the developer. In the preceding example, observe that routing can be easily performed in Express.js. Express.js thus enables developers to perform server-side backend development with Node.js without the limitations. Express.js is also the underlying library for several other popular Node.js web frameworks.

In the next video, I will explain more about how to create a server-side rendering web application using Express.js.

Video 4: Server-Side Web Application Development with Express.js

In this video, you will learn how to create static content in a server-side rendering web application using Express.js and JavaScript. Let's begin with some basic routing concepts. Routing determines how a web application responds to a client's HTTP request that is sent to a particular endpoint. An endpoint is a combination of a Uniform Resource Identifier, or URI and a specific HTTP request method or verb.

An URI consists of the HTTP protocol specifier, domain name or IP address of the web server and followed by a path within the web application. Let's look at a simple example of an endpoint, GET <http://localhost:3000/car>. In this example, HTTP is the protocol specifier, localhost:3000 points to the web server running on port 3000 of your local computer and the path is /car. The root path of the web application is simply denoted by a forward slash.

Within an Express.js web application, endpoints are defined as routes. Each route consists of a handler function that is associated with a path without the protocol specifier and server name and a HTTP method. Thus, a handler function is executed when the path is matched with the corresponding HTTP method. The `app.METHOD` function of Express.js is used to define routing. HANDLER is used to designate a JavaScript function that is executed when a route is matched. The string return value of the function becomes the HTML content displayed in a web browser. Express.js supports methods that correspond to all HTTP request methods. `app.all()` is a special routing method that is used to load a handler function at a path for all HTTP request methods. But do take note that for a web application serving HTML content, we typically use only the GET and POST methods. The other methods are used for API endpoints, which will be discussed in a future video.

Let's look at a simple example of routing in Express.js. In this example, there are three routes being defined. The first route is for the root path. The next two routes are for the /car and /book paths. Sending a GET `http://localhost:3000/car` request from either a web browser or a testing tool such as Postman will display the streamed data returned by the handler function as HTML content. In Express.js, route paths can be strings, string patterns, or regular expressions. For example, the path pattern shown here will match `abcd`, `abccd`, `abbbcd`, etc. This mechanism provides flexibility to route definition. If you need to define multiple routes for the same path but with different HTTP methods, a more concise syntax of route chaining can be used with the `app.route` function of Express.js. In this example, we define two endpoints for the same /car route but with two different HTTP methods of GET and POST.

Let's move on to find out how to pass data to a web application using parameters. This is a very useful technique for customising the content generated by a web application. For example, you might define a route `/product` to display a list of product items. Using this single route, we can pass in the required product category to the web application so that it can render the required content. This is more efficient than defining two separate routes of /car and /book for example.

There are two types of parameters that can be defined in Express.js. The first type is route parameter, which uses named segments in the path to capture the value specified at their respective position in the path. For each value, the key is the name of the route parameter specified in the path. Route parameters are defined using the `/:param` convention. This example shows the category route parameter defined in the `/product` route. The second type of parameter is the query string parameter. This type of parameter is appended to the end of the path after the question mark special character using this convention. This example shows the category query string parameter defined in the `/product` route.

In general, route parameter is preferred if search engine optimisation is important as search engines rank a dynamic path containing query string parameter lower. Do also take note that the data type of parameter values default to string. If you need to manipulate a number, you should perform the required type casting operation. Methods in the response object or `res` can send response to the client or web browser and terminate the request-response cycle.

A web application normally uses the `res.send()` method to send back the HTML content as you would have observed in the preceding example codes. `res.sendFile()` can also be used to send a file such as PDF documents back to the web browser.

Finally, an Express.js web application can handle conversational state using the session middleware. A middleware function is a special function that has access to the request and response objects for performing intermediate processing before the route's handler function is invoked. To save data in the session middleware, you can use the `req.session` object together with the required property name. A new property defaults to undefined and should be initialised before the first use. Express.js uses a session ID to identify each unique user and this session ID is stored in a cookie. The session ID then is sent back to the server with each request to correctly identify the user. The actual session data is stored on the server.

In this example, a simple counter is stored in the session of each user such that the counter's value is uniquely incremented. Observe that the two browser windows are showing values of different counters. Common data that are stored in the session include login information and shopping cart data. This wraps up our discussion on creating server-side rendering web applications with Express.js.

Video 5: Using the Pug Template Engine with Express.js

Thus far, you have learnt how to create a server-side rendering web application using Express.js. As you have observed the basic programming model is really easy. You just need to define one or more endpoints or routes. Each route is a combination of a path and HTTP method that is associated with a handler function. The handler function is just a JavaScript function that returns a string value. Whatever string value that is returned by the handler function gets rendered by the web browser as HTML content. However, you will also realise that writing and formatting HTML content in a JavaScript function is rather tedious and unintuitive. This becomes a complex task as the amount of HTML content increases. It gets even harder when you need to intersperse static HTML with dynamic HTML.

Fortunately, Express.js supports the use of templating. Templating allows us to define the HTML content in separate files known as templates. Templates can then be easily returned by handler functions to a web browser for display to the user. Express.js supports many different template engines, but Pug is the default one. Pug simplifies the process of writing HTML tags by reducing traditional HTML syntax. More specifically, it uses indentation to represent HTML structure, does not require you to write the angle brackets and omits the closing tag. Pug templates end with the `.pug` file extension. This example demonstrates the differences between the traditional HTML syntax and the Pug syntax. The source file on the left uses traditional HTML syntax and the source file on the right uses Pug syntax. Observe that the Pug syntax is certainly more simplified. The Pug template engines will compile the Pug source file into a JavaScript function that returns the content in traditional HTML syntax so that a web browser understands how to display the content. To integrate the Pug template engine with your Express.js web application, you just need to create the Pug templates and placed them inside a "views" subfolder.

In the route handler functions, replace the `res.send` method with the `res.render` method. In this example, the handler function of the default route continue to use the `res.send` method. But the route handler function of the `/car` route uses the `res.render` method to render the `car.pug` template as HTML back to the web browser. Dynamic data or variables from the handler function can be passed into the Pug template for rendering using the interpolation with the `#{}` notation.

Interpolation also allows JavaScript expressions to be rendered. Examples of such dynamic data include parameters passed in by the user or data retrieved from a database. In this example, the name and age variables are passed into the Pug template for rendering. Observe that the name and age are defined within a JavaScript object in the handler function.

The Pug template engine is very versatile and powerful. It provides support for conditional rendering of content using the if statement and iterative rendering of content using the each loop. The each loop can be used to render JavaScript arrays. In this example, we passed a Boolean variable and a JavaScript array of objects for rendering inside a Pug template in conjunction with interpolation. The array of objects is hardcoded in our source file. In a real-world web application, the array of objects can be retrieved from a database. To include static assets such as images and stylesheets into Pug templates, you need to first define the subfolder containing these static assets. Then use the corresponding tag name such as img for images in the Pug template to render them. In this example, we render an image placed within the images subfolder of the express.js web application.

As you would have observed by now, working with a template engine such as Pug certainly makes it much easier to create content in your web application. Go on and explore more about Pug!

Video 6: Creating Database-Driven Web Application with Express.js

A server-side rendering web application created using Express.js can be used to serve both static and dynamic HTML content. Thus far, the dynamic contents in the preceding videos are based on data that are hardcoded within the JavaScript. But the biggest business value of a web application is derived by integrating it with a database.

Let's examine a common example of an e-commerce web application. Within an e-commerce application, we can create just a few template pages to render the product listings and product details of hundreds and thousands of items by retrieving the data from a database. Whenever a customer checks out his or her shopping cart, the data can be stored in a database to facilitate order fulfilment by the business and order tracking by the customer. These are just some basic use cases. Integrating a web application with a database opens a whole new world of business opportunities including database marketing, customer relationship management and many more.

Express.js provides a comprehensive database integration capability to connect a web application to a database in a very easy manner. You just need to load an appropriate Node.js driver for your database and then utilise a set of database manipulation functions to read and write data to and from the database. Most of these functions require you to understand how to write a Structured Query Language or SQL statements such as a select query and an insert statement.

In this video, we'll use SQLite, a popular serverless relational database management system to facilitate our discussion for simplicity. The concepts and skills covered in this video with SQLite are equally applicable to any server-based relational database management systems such as MySQL. The basic programming model of integrating a web application with a database is very easy and consist of just two steps. First, you need to open a connection to the database. Second, call the required database manipulation functions with the corresponding SQL statements.

Let's examine a data retrieval example using a simple database that contains one table for the car records. In this example, we use the db.all function to retrieve multiple car records with a select query. Observe that the data returned from the query is already in the format of a JavaScript object. In this case, we have a JavaScript array of objects. Thus, it is very easy to manipulate the data returned by a query in your Express.js web application.

If you're expecting only one result to be returned from a select query, you can also use the `db.get` function. In the next example, we will pass the results of the query into a Pug template for rendering into a dynamic web page. To perform write operations with a database, you can use the `db.run` function to execute the insert, update and delete SQL statements as required. These SQL statements often require you to pass in parameters such as the data to be inserted or updated. The `db.run` function allows you to specify placeholders in a SQL statement and pass in the actual parameter values as function arguments, array, or named parameters. This example demonstrates how to execute the insert SQL statement using all three approaches. Observe that it is really easy to work with a database in an Express.js web application using the database integration capability.

Do explore more about the data manipulation functions provided by Express.js. Express.js also supports object mapping frameworks such as Sequelize. Sequelize is a popular object relational mapping framework that allows you to work with data stored in a relational database easily using only JavaScript code without the hassle of the SQL statements. Sequelize also makes it easy to define the data models in your web application and automatically generate the required database tables. Do find out more about Sequelize or other object mapping frameworks if you're interested.

Video 7: Introduction to Flask

Recall that backend development mainly involves server-side rendering web applications and Application Programming Interface or API endpoints. This video provides you with an overview of developing server-side rendering web application in Python using the Flask framework. We will discuss how to develop API endpoints with Flask in a future module. Other than Flask, there are many other web frameworks in python such as Django and Pyramid. But Flask is a lightweight framework that is both easy to learn and use. So let's get started!

The Flask framework exhibits several characteristics that make it ideal for web application development. First, Flask supports routing, which allows multiple routes to be easily defined for different sections of a web application. In addition, each route can be used with one or more HTTP methods, usually GET and POST. Second, Flask supports templating through the Jinja2 template engine. This allows HTML tags to be easily written and formatted in template files for rendering into web pages. In addition, static assets such as images and stylesheets can also be readily incorporated into the template files. Third, Flask provides support for conversational state management. This allows information specific to a user to be stored from one request to the next. Finally, Flask offers good database integration capability for reading and writing data to and from a database. In fact, you can also use object relational mapping frameworks such as SQL Alchemy. Flask can also be used for more advanced use cases such as file uploads, cookies handling, redirections and errors handling.

In gist, Flask is a modern web application development framework that significantly enhances the productivity of developers. You would probably have observed by now that the Flask framework resembles the Express.js framework. Indeed, most modern web application development frameworks share similar characteristics and capabilities. Once you have become conversant in web application development with one framework, it is relatively easy to learn other frameworks.

Video 8: Server-Side Web Application Development with Flask

In this video, you will learn how to create static content in a server-side rendering web application using Flask and Python. In Flask, routing is defined using the route decorator. More specifically, the route decorator binds a Python function to a path within the web application. Whatever string value returns by this Python function will be rendered by the web browser as HTML content.

In this example, we define three different routes altogether, namely the default route or root path and the /car and /book routes. Sending a GET request to this address, for example, from either a web browser or a testing tool, such as Postman will display the string value returned by the car() Python function as HTML content. By default, a route only responds to HTTP GET requests. The methods parameter of the route decorator can be used to specify different HTTP methods to be handled by the designated Python function.

The GET and POST methods are more commonly used for web application. In this example, we define the default route to respond to both GET and POST requests. Within the index() Python function, we can differentiate between the processing and return value of each method using the request.method property. Calling the default route with GET and POST will result in a successful response or HTTP 200 status albeit with different HTML content. However, a PUT request will result in a HTTP 405 error, which indicates that the PUT method is not allowed or not supported.

We can also pass data to a Flask web application using parameters. Both path parameters and query string parameters are supported in Flask. To define path parameters, the required variable sections should be added to a path using the <variable_name> notation. The designated Python function will then be able to receive the value of variable as a keyword argument of the same name. Recall that HTTP request parameters are string by default. You can use an optional converter to change the data type of a parameter value, if required. In this example, we define two path parameters, name and age. Observe that age is converted to an integer. Query string parameters do not need to be defined explicitly. You can retrieve the values of query string parameters using the request.args.get() method. In this example, we define name and age as query string parameters. Observe that the output returned by the Python function is similar to that in the path parameters example.

A Flask web application can also serve static files such as CSS stylesheets and images from the designated static subfolder. Static files in the static subfolder will be made available at the path /static. You can use the url_for() function to generate the required paths to the static files. In this example, the CSS class special that renders the foreground colour of the level one heading in red is defined in the external stylesheet default.css. This stylesheet together with the image file are placed inside the static subfolder. Observe the use of the Python tripe quotes notation to define a multiline string literal representing the HTML content.

Finally, a Flask web application can handle conversational state using the session object. The session object is essentially a Python dictionary or dict data structure that contains key-value pairs of session variables and their associated values. A session with each unique user is assigned a session ID. The assignment operator is used to set a new session variable. The session.pop() function is used to remove a session variable when it is no longer required. The session.get() function allows a default value to be returned when the corresponding key is not in the session object.

In this example, a simple counter is stored in the session of each user such that the counter's value is uniquely incremented. Observe that the two browser windows are showing values of different counters. This wraps up our discussion on creating sever-side rendering web application with Flask.

Video 9: Using the Jinja2 Template Engine with Flask

In the preceding video, you have learnt how to write and format HTML content inside a Python function and return the content as a string value to a web browser for display. Even though Python supports the triple quotes notation for multiline string literal, it is still tedious and unintuitive to create HTML content in this manner. Fortunately, Flask also supports the rendering of template files.

Template files can contain both static and dynamic HTML content. By default, Flask is configured to use the Jinja2 template engine. The Jinja2 template engine uses the traditional HTML syntax. In other words, you can simply markup the required content using HTML tags. To integrate the Jinja2 template engine with your Flask web application, you just need to create the templates and place them inside a "templates" subfolder. In the designated Python function, instead of returning a raw string value, you will now return the output of the `render_template()` function. A Jinja2 template file does not require any special file extension. For simplicity, we'll use the `.html` file extension for our templates.

In this example, we will define a `/car` route to render `car.html` back to the web browser. Observe that the templating process in Flask with Jinja2 is similar to that of Express.js with Pug. Dynamic data or variables from the Python function can be passed into the Jinja2 template for rendering using interpolation with the `{{}}` bracket notation. In this example, the `name` and `age` variables are passed into the Jinja2 template for rendering. Observe that `name` and `age` are defined within the `render_template()` function as named parameters.

The Jinja2 template engine provides support for conditional rendering of content using the `if` statement and iterative rendering of content using the `for` loop. The `for` loop can be used to iterate over the elements of a list or the keys of a dictionary. In this example, we passed in a Boolean variable and a Python list of dict for rendering inside a Jinja2 template in conjunction with interpolation. Observe that the list of dict is hardcoded in our source file. In a real world web application, the list of dict can be retrieved from a database. To include static assets such as images and stylesheets into Jinja2 templates, you can directly reference static files in the static subfolder. In this example, observe the use of relative path to reference the static files. This approach is similar to the use of the `url_for()` function.

As you would have observed by now, working with a template engine such as Jinja2 in Flask is similar to the Pug template engine in Express.js. In general, the use of templates makes it much easier to create content in your web application.

Video 10: Creating Database-Driven Web Application with Flask

Python provides a comprehensive set of libraries for working with relational database management systems. Most major vendors such as SQLite, MySQL, and Postgres are supported. To perform database integration between a Flask web application and a relational database, you just need to install the required driver that is based on the Python DB-API 2.0 interface. In this video, we will also be using SQLite to facilitate our discussion for simplicity. The concepts and skills covered in this video with SQLite are equally applicable to any server-based relational database management systems such as MySQL.

Similar to the database integration in an Express.js web application, the basic programming model of integrating a Flask web application with a SQLite database is very easy and consists of just two steps. First, you need to open a connection to the database. Second, call the required database manipulation functions with the corresponding SQL statements.

Let's examine a data retrieval example using a simple database that contains one table for the car records. In this example, we use the database cursor object to execute a select query and fetch all the result rows. Observe that the data returned from the query is already in the format of a Python list of tuple data structure. Thus, it is very easy to manipulate the data returned by a query in your Flask web application. If you're expecting only one result to be returned from a select query, you can also use the `fetchone()` function instead of the `fetchall()` function.

In the next example, we will pass the results of the query into a Jinja2 template for rendering into a dynamic web page. To perform right operations with a database, you can use the cursor execute() function to execute the insert, update and delete SQL statements as required. These SQL statements often require you to pass in parameters such as the data to be inserted or updated. You can pass in a tuple of values to the cursor execute() function to perform this task in conjunction with the ? placeholder notation. Alternatively, you can use the python string formatting technique.

This example demonstrates how to execute the insert SQL statement. Observe that it is easy to work with a database in a Flask web application. Python also supports object relational mapping frameworks such as SQLAlchemy. Do explore more about the various advanced database integration capability of Python and Flask.

Video 11: Module Summary

In this module, you have learnt how to create server-side rendering web applications using oth JavaScript and Python. In JavaScript, we have used the Express.js framework and in Python, we have used the Flask framework. These modern frameworks provide us with best-in-class capabilities to create dynamic web pages using data stored in a database in conjunction with templating. These are important skills that you would find useful in creating full-stack AI solutions. In the next module, you will extend the concepts and skills learnt in this module to create API endpoints. API endpoints allow you to return general data instead of HTML content to the client. This increases the flexibility of the AI solutions in supporting client-side rendering web applications, mobile applications and even Internet of Things or IoT applications.

THE END